

Tidal Data: Assessing factors driving marbled murrelet distribution and density in Puget Sound using hierarchical distance sampling

Code by S.J. Gillman

Description and Sources

1. NOAA Tide & Currents Data

- This can be downloaded via [API](#) with description of [CO-OPS API For Data Retrieval](#).

Downloading data info

Station ID A 7 character station ID, or a currents station ID. Specify the station ID with the “station=” parameter. Examples: station=9414290 (water level / met station) station=cb1401 (currents station) [Station listings](#) or viewed on a map at Tides & Currents Station Map

Date & Time The API understands several parameters related to date ranges. All dates can be formatted as follows: yyyyMMdd, yyyyMMdd HH:mm, MM/dd/yyyy, or MM/dd/yyyy HH:mm

Stations for Puget Sound: - Port Townsend - StationID = “9444900” - Latitude: 48.11170 - Longitude: -122.75800

- Bremerton
 - StationID = “9445958”
 - Latitude: 47.56170
 - Longitude: -122.62300
- Tacoma
 - StationID = “9446484”
 - Latitude: 47.26667
 - Longitude: -122.41333
- Seattle
 - StationID = “9447130”
 - Latitude: 47.60500
 - Longitude: -122.33800
- Friday Harbor
 - StationID = “9449880”
 - Latitude: 48.54670
 - Longitude: -123.01000

Monthly Means The monthly mean commands (although I have datum as MHW) give you all the monthly values for that station: - MHW: Mean High Water - MSL: Mean Sea Level - MLW: Mean Low Water - GT: Great Diurnal Range - Inferred: A flag that when set to 1 indicates that the water level value has been inferred

2. Regional Zoning

- CO-OPS_Regional_Zoning.gpkg is the [Zoning](#) used to determine the tidal information for areas outside of the station locations.

Required Packages

```
library(sf)
library(tidyverse)
library(ggplot2)
```

Required Files Previously Made or Downloaded

- puget_poly.gpkg
- grid2km_hex.gpkg
- reduced_area.gpkg
- all_tracks_2026_final.gpkg
- lat_long_times.rds which are individual gps points with times for each trackline.

API Downloads

Port Townsend

monthly means https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20120131&end_date=20241231&station=9444900&product=monthly_mean&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2012 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20120131&end_date=20121231&station=9444900&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2013 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20130131&end_date=20131231&station=9444900&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2014 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20140131&end_date=20141231&station=9444900&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2015 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20150131&end_date=20151231&station=9444900&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2016 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20160131&end_date=20161231&station=9444900&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2017 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20170131&end_date=20171231&station=9444900&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2018 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20180131&end_date=20181231&station=9444900&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2019 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20190131&end_date=20191231&station=9444900&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2020 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20200131&end_date=20201231&station=9444900&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2021 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20210131&end_date=20211231&station=9444900&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2022 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20220131&end_date=20221231&station=9444900&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2023 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20230131&end_date=20231231&station=9444900&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2024 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20240131&end_date=20241231&station=9444900&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

Bremerton

monthly means https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20120131&end_date=20241231&station=9445958&product=monthly_mean&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2012 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20120131&end_date=20121231&station=9445958&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2013 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20130131&end_date=20131231&station=9445958&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2014 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20140131&end_date=20141231&station=9445958&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2015 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20150131&end_date=20151231&station=9445958&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2016 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20160131&end_date=20161231&station=9445958&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2017 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20170131&end_date=20171231&station=9445958&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2018 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20180131&end_date=20181231&station=9445958&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2019 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20190131&end_date=20191231&station=9445958&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2020 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20200131&end_date=20201231&station=9445958&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2021 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20210131&end_date=20211231&station=9445958&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2022 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20220131&end_date=20221231&station=9445958&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2023 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20230131&end_date=20231231&station=9445958&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2024 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20240131&end_date=20241231&station=9445958&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

Tacoma

monthly means https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20120131&end_date=20241231&station=9446484&product=monthly_mean&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2012 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20120131&end_date=20121231&station=9446484&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2013 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20130131&end_date=20131231&station=9446484&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2014 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20140131&end_date=20141231&station=9446484&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2015 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20150131&end_date=20151231&station=9446484&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2016 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20160131&end_date=20161231&station=9446484&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2017 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20170131&end_date=20171231&station=9446484&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2018 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20180131&end_date=20181231&station=9446484&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2019 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20190131&end_date=20191231&station=9446484&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2020 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20200131&end_date=20201231&station=9446484&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2021 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20210131&end_date=20211231&station=9446484&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2022 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20220131&end_date=20221231&station=9446484&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2023 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20230131&end_date=20231231&station=9446484&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2024 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20240131&end_date=20241231&station=9446484&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

Seattle

monthly means https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20120131&end_date=20241231&station=9447130&product=monthly_mean&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2012 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20120131&end_date=20121231&station=9447130&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2013 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20130131&end_date=20131231&station=9447130&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2014 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20140131&end_date=20141231&station=9447130&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2015 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20150131&end_date=20151231&station=9447130&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2016 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20160131&end_date=20161231&station=9447130&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2017 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20170131&end_date=20171231&station=9447130&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2018 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20180131&end_date=20181231&station=9447130&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2019 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20190131&end_date=20191231&station=9447130&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2020 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20200131&end_date=20201231&station=9447130&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2021 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20210131&end_date=20211231&station=9447130&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2022 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20220131&end_date=20221231&station=9447130&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2023 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20230131&end_date=20231231&station=9447130&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2024 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20240131&end_date=20241231&station=9447130&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

Friday Harbor

monthly means https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20120131&end_date=20241231&station=9449880&product=monthly_mean&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2012 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20120131&end_date=20121231&station=9449880&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2013 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20130131&end_date=20131231&station=9449880&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2014 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20140131&end_date=20141231&station=9449880&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2015 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20150131&end_date=20151231&station=9449880&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2016 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20160131&end_date=20161231&station=9449880&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2017 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20170131&end_date=20171231&station=9449880&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2018 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20180131&end_date=20181231&station=9449880&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2019 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20190131&end_date=20191231&station=9449880&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2020 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20200131&end_date=20201231&station=9449880&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2021 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20210131&end_date=20211231&station=9449880&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2022 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20220131&end_date=20221231&station=9449880&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2023 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20230131&end_date=20231231&station=9449880&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv

hourly 2024 https://api.tidesandcurrents.noaa.gov/api/prod/datagetter?begin_date=20240131&end_date=20241231&station=9449880&product=hourly_height&datum=MHW&time_zone=gmt&units=metric&format=csv ***

Regional Zoning

[Zoning](#) used to determine the tidal information for areas outside of the station locations.

```
noaa_zones <- st_read("../GIS_COVS/CO-OPS_Regional_Zoning.gpkg")
```

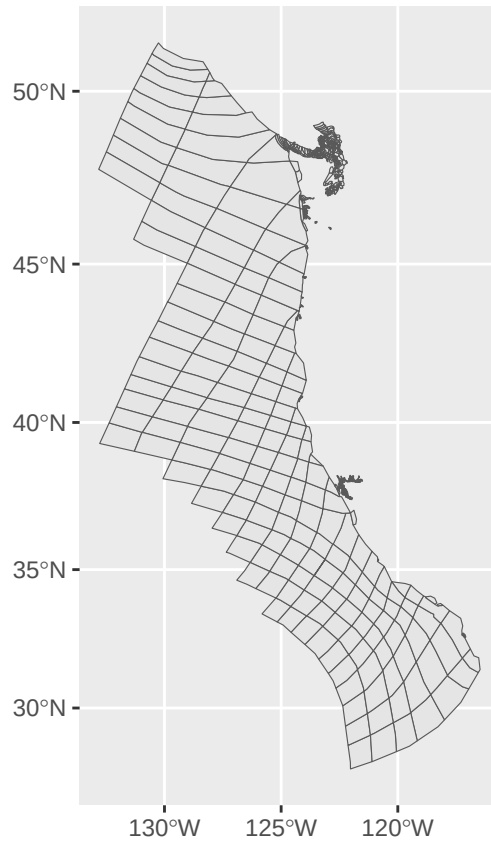
```
head(noaa_zones)
```

```
## Simple feature collection with 6 features and 7 fields
## Geometry type: MULTIPOLYGON
## Dimension:      XYZM
## Bounding box:   xmin: -13754000 ymin: 5786523 xmax: -13676920 ymax: 5824648
## z_range:        zmin: 0 zmax: 0
## m_range:        mmin: NA mmax: NA
## Projected CRS: WGS 84 / Pseudo-Mercator
##   ControlStn ATCorr   RR ControlStn2 ATCorr2 RR2 DataProv
## 1    9439201   -72 1.34          <NA>    NA   NA   CO-OPS
## 2    9439201   -66 1.32          <NA>    NA   NA   CO-OPS
## 3    9439201   -54 1.26          <NA>    NA   NA   CO-OPS
## 4    9439201   -78 1.37          <NA>    NA   NA   CO-OPS
## 5    9439201   -60 1.29          <NA>    NA   NA   CO-OPS
## 6    9440569    -6 1.02          <NA>    NA   NA   CO-OPS
##
##                               SHAPE
## 1 MULTIPOLYGON ZM (((-1368033...
## 2 MULTIPOLYGON ZM (((-1367794...
## 3 MULTIPOLYGON ZM (((-1368087...
## 4 MULTIPOLYGON ZM (((-1368433...
## 5 MULTIPOLYGON ZM (((-1367692...
## 6 MULTIPOLYGON ZM (((-1374997...
```

```
st_crs(noaa_zones)
```

```
## Coordinate Reference System:
##   User input: WGS 84 / Pseudo-Mercator
##   wkt:
## PROJCRS["WGS 84 / Pseudo-Mercator",
##     BASEGEOGCRS["WGS 84",
##       ENSEMBLE["World Geodetic System 1984 ensemble",
##         MEMBER["World Geodetic System 1984 (Transit)"],
##         MEMBER["World Geodetic System 1984 (G730)"],
##         MEMBER["World Geodetic System 1984 (G873)"],
##         MEMBER["World Geodetic System 1984 (G1150)"],
##         MEMBER["World Geodetic System 1984 (G1674)"],
##         MEMBER["World Geodetic System 1984 (G1762)"],
##         MEMBER["World Geodetic System 1984 (G2139)"],
##         MEMBER["World Geodetic System 1984 (G2296)"],
##       ELLIPSOID["WGS 84",6378137,298.257223563,
##         LENGTHUNIT["metre",1]],
##       ENSEMBLEACCURACY[2.0]],
##     PRIMEM["Greenwich",0,
##       ANGLEUNIT["degree",0.0174532925199433]],
##     ID["EPSG",4326]],
##     CONVERSION["Popular Visualisation Pseudo-Mercator",
##       METHOD["Popular Visualisation Pseudo Mercator",
##         ID["EPSG",1024]],
##       PARAMETER["Latitude of natural origin",0,
##         ANGLEUNIT["degree",0.0174532925199433],
##         ID["EPSG",8801]],
##       PARAMETER["Longitude of natural origin",0,
##         ANGLEUNIT["degree",0.0174532925199433],
##         ID["EPSG",8802]],
##       PARAMETER["False easting",0,
##         LENGTHUNIT["metre",1],
##         ID["EPSG",8806]],
##       PARAMETER["False northing",0,
##         LENGTHUNIT["metre",1],
##         ID["EPSG",8807]]],
##     CS[Cartesian,2],
##       AXIS["easting (X)",east,
##         ORDER[1],
##         LENGTHUNIT["metre",1]],
##       AXIS["northing (Y)",north,
##         ORDER[2],
##         LENGTHUNIT["metre",1]],
##     USAGE[
##       SCOPE["Web mapping and visualisation."],
##       AREA["World between 85.06°S and 85.06°N."],
##       BBOX[-85.06,-180,85.06,180]],
##     ID["EPSG",3857]]
```

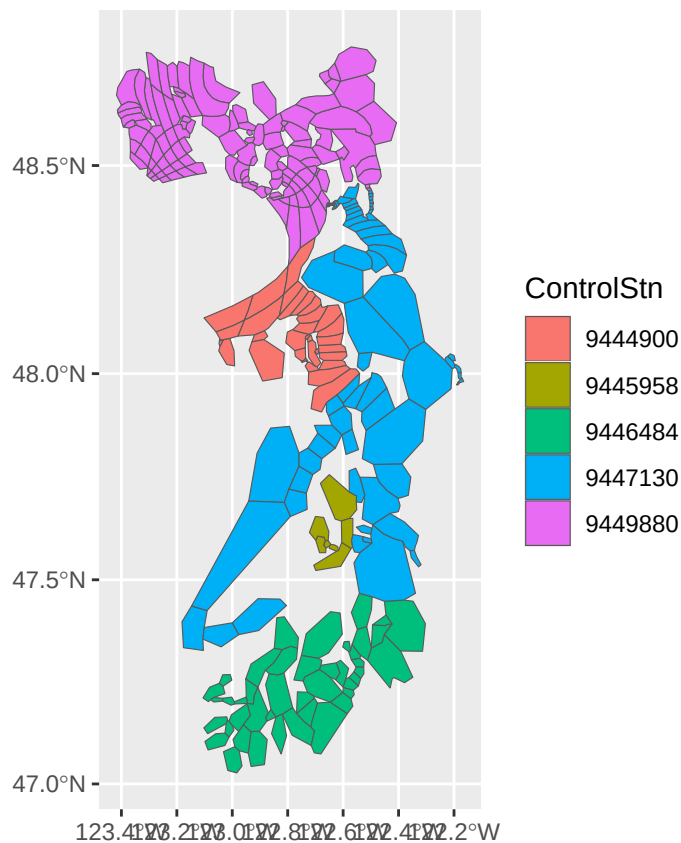
```
ggplot()+
  geom_sf(data=noaa_zones)
```



```
## filter to only the stations we need

puget_zones <- noaa_zones %>%
  filter(ControlStn == 9444900 | ControlStn2 == 9444900 |
         ControlStn == 9445958 | ControlStn2 == 9445958 |
         ControlStn == 9446484 | ControlStn2 == 9446484 |
         ControlStn == 9447130 | ControlStn2 == 9447130 |
         ControlStn == 9449880 | ControlStn2 == 9449880)

ggplot()+
  geom_sf(data=puget_zones, aes(fill = ControlStn))
```



Load Required

```
puget <- st_read("../GIS_COVS/puget_poly.gpkg")
studyarea <- st_read("../GIS_COVS/reduced_area.gpkg")
hex_grids <- st_read("../GIS_COVS/grid2km_hex.gpkg")
tracklines <- st_read("../GIS_COVS/all_tracklines_2026_final.gpkg")
lat longs <- readRDS("../GIS_COVS/lat_long_times.rds")
```

Monthly Averages

```
folder_path <- "../GIS_COVS/NOAA"

csv_files <- list.files(folder_path, pattern = "\\..csv$", full.names = TRUE)

get_suffix <- function(file_path) {
  file_name <- basename(file_path) # Extract just the file name
  file_name <- sub("\\..csv$", "", file_name) # Remove the ".csv" extension
  # Extract last two characters
  return(substr(file_name, nchar(file_name) - 1, nchar(file_name)))
}

# Extract suffixes
```

```

suffixes <- sapply(csv_files, get_suffix)

# Load all CSVs into a single data frame and add the suffix as a new column
tide_obs <- do.call(rbind, lapply(seq_along(csv_files), function(i) {
  data <- read.csv(csv_files[i])      # Read the CSV file
  data$station <- suffixes[i]        # Add the suffix as a new column
  return(data)
}))

names(tide_obs) <- tolower(names(tide_obs))

## Add station IDs
tide_obs <- tide_obs %>%
  mutate(ControlStn = ifelse(station == "FH", 9449880,
                             ifelse(station == "BM", 9445958,
                             ifelse(station == "PT", 9444900,
                             ifelse(station == "TA", 9446484, 9447130)))))

```

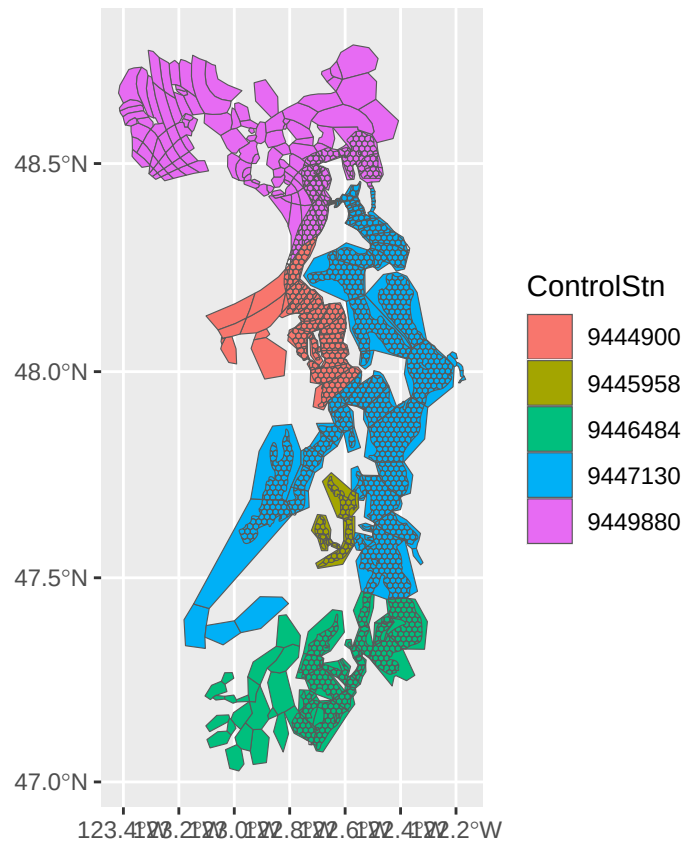
Determine which zone segment the grid is in

```

hex_grid_transformed <- st_transform(hex_grids, crs= st_crs(puget_zones))

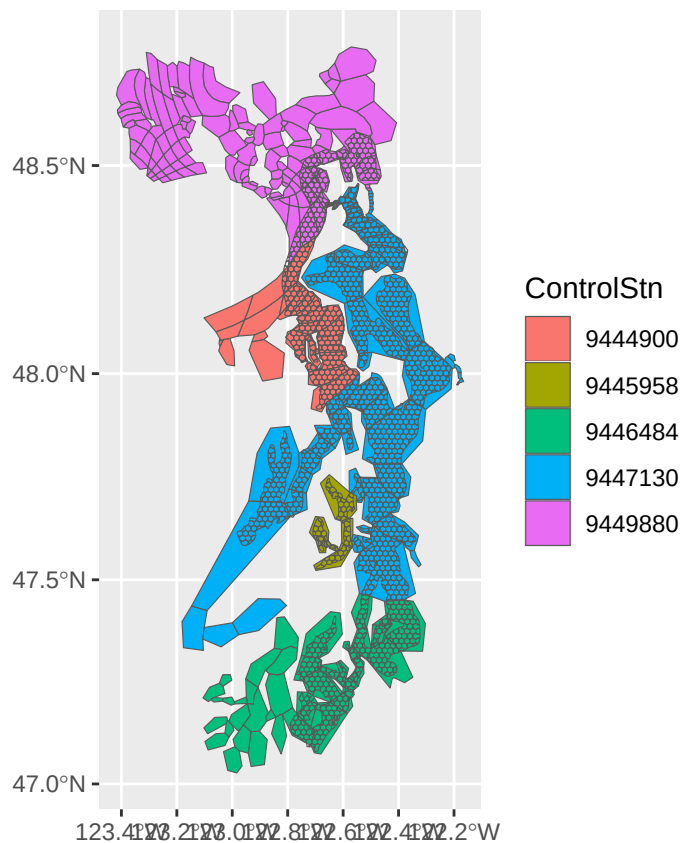
ggplot()+
  geom_sf(data=puget_zones, aes(fill = ControlStn))+
  geom_sf(data = hex_grid_transformed, fill =NA)

```



```
puget_zones <- st_zm(puget_zones, drop = TRUE)

ggplot()+
  geom_sf(data=puget_zones, aes(fill = ControlStn))+
  geom_sf(data = hex_grid_transformed, fill =NA)
```



```
hex_zones <- st_intersection(hex_grid_transformed,puget_zones) %>%
  st_drop_geometry() %>%
  distinct(ID,ControlStn)
```

```
hex_zones %>%
  group_by(ID) %>%
  count() %>%
  filter(n>1)
```

```
## # A tibble: 29 x 2
## # Groups:   ID [29]
##   ID      n
##   <chr> <int>
## 1 G110    2
## 2 G1213   2
## 3 G125    2
## 4 G1287   2
## 5 G1324   2
## 6 G1357   2
## 7 G1385   2
## 8 G144    2
## 9 G164    2
## 10 G192   2
## # i 19 more rows
```

Making the corrected values by merging the zonal and obs data

```

puget_zonobs <- hex_zones %>%
  mutate(ControlStn = as.character(ControlStn)) %>%
  full_join(tide_obs %>%
    mutate(ControlStn = as.character(ControlStn)) %>%
    dplyr::select(c(ControlStn,station,year,month,
                    gt,mhw,msl,mlw)), by = "ControlStn") %>%
  mutate(mon = month.abb[month]) %>%
  mutate(moneyear = paste0(mon,"-",year)) %>%
  group_by(ID,moneyear) %>%
  dplyr::summarize(mhwRR =mean(mhw,na.rm=T),
                  gtRR =mean(gt,na.rm=T),
                  mslRR =mean(msl,na.rm=T),
                  mlwRR =mean(mlw,na.rm=T),.groups="drop") %>%
  ungroup() %>%
  distinct()

hex_grids %>%
  st_drop_geometry() %>%
  filter(!ID %in% puget_zonobs$ID)

```

```

## [1] area ID numid
## <0 rows> (or 0-length row.names)

```

```

saveRDS(puget_zonobs,"GIS_COVS/grid_tides.rds")

```

Transect Data

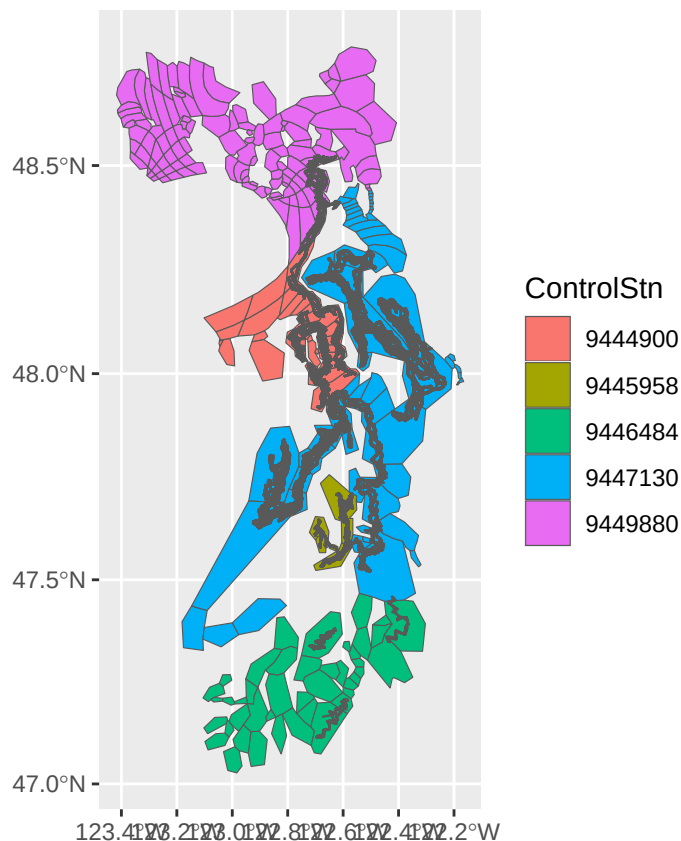
Determine which zone segment the grid is in

```

track_transformed <- st_transform(tracklines, crs= st_crs(puget_zones)) %>%
  st_buffer(dist = 215)

ggplot()+
  geom_sf(data=puget_zones, aes(fill = ControlStn))+
  geom_sf(data = track_transformed, fill =NA)

```



```
track_zones <- st_intersection(track_transformed,puget_zones) %>%
  st_drop_geometry() %>%
  distinct(trip_date,date, ControlStn, RR,ATCorr) %>%
  mutate(year = str_sub(date, 1,4),
         month = month(date))
```

Making the corrected values by merging the zonal and obs data

```
puget_zonobs_track <- track_zones %>%
  mutate(ControlStn = as.character(ControlStn)) %>%
  left_join(tide_obs %>%
    mutate(ControlStn = as.character(ControlStn),
           year = as.character(year)) %>%
    dplyr::select(c(ControlStn,station,year,
                    month,gt,mhw,msl,mlw)),
    by = c("ControlStn","year","month")) %>%
  mutate(mon = month.abb[month]) %>%
  mutate(moneyear = paste0(mon,"-",year)) %>%
  group_by(trip_date, moneyear) %>%
  dplyr::summarize(mhwRR =mean(mhw,na.rm=T),
                  gtRR =mean(gt,na.rm=T),
                  mslRR =mean(msl,na.rm=T),
                  mlwRR =mean(mlw,na.rm=T),.groups="drop") %>%
  ungroup() %>%
  distinct()
```

```

tracklines %>%
  st_drop_geometry() %>%
  filter(!trip_date %in% puget_zonobs_track$trip_date)

## [1] trip_date    date            total_length
## <0 rows> (or 0-length row.names)

saveRDS(puget_zonobs_track, "GIS_COVS/track_tides.rds")

```

Observation Level Tide

```

folder_path <- "../GIS_COVS/NOAA/hourly_tide"

csv_files <- list.files(folder_path, pattern = "\\*.csv$", full.names = TRUE)

get_suffix <- function(file_path) {
  # Extract just the file name
  stationid <- gsub(".*__(\\d+)__.*", "\\1", basename(file_path))
  return(stationid)
}

# Extract suffixes
suffixes <- sapply(csv_files, get_suffix)

# Load all CSVs into a single data frame and add the suffix as a new column
tide_hour <- do.call(rbind, lapply(seq_along(csv_files), function(i) {
  data <- read.csv(csv_files[i]) # Read the CSV file
  data$station <- suffixes[i] # Add the suffix as a new column
  return(data)
})))

names(tide_hour) <- tolower(names(tide_hour))

## split date time

tide_hour_date <- tide_hour %>%
  mutate(date.time = as.POSIXct(date.time, format = "%Y-%m-%d %H:%M"),
         date = as.Date(date.time),
         time = format(date.time, "%H:%M")) %>%
  filter(!is.na(water.level))

```

Linking to Observations

```

track_times <- lat longs %>%
  full_join(track_zones, by = c("trip_date", "date")) %>%
  group_by(trip_date, date, ControlStn, RR, ATCorr) %>%
  summarize(start_time = min(time), end_time = max(time),
            .groups = "drop") %>%

```

```

ungroup() %>%
filter(trip_date != "Z1S2P31N-AR1 2013-06-05")

tide_hour_filtered <- track_times %>%
  mutate(date2 = as.character(date),
         start_time = hms::as_hms(
           parse_date_time(if_else(is.na(start_time),
                                   "08:00:00", start_time), "HMS")),
         end_time = hms::as_hms(
           parse_date_time(if_else(is.na(end_time),
                                   "18:30:00", end_time), "HMS"))) %>%
  left_join(tide_hour_date %>%
            rename(ControlStn = station) %>%
            mutate(date2 = as.character(date)),
            by = c("ControlStn", "date2")) %>%
  rename(date = date.x) %>%
  mutate(time_hms = hms::as_hms(ifelse(nchar(time) <= 5,
                                       paste0(time, ":00"), time))) %>%
  filter((time_hms >= start_time & time_hms <= end_time)) %>%
  group_by(trip_date, date) %>%
  summarise(survey_tide = mean(water.level * RR, na.rm = TRUE),
            .groups="drop") %>%
  ungroup() %>%
  distinct(trip_date, date, survey_tide)

length(unique(tide_hour_filtered$trip_date))

```

```
## [1] 2588
```

Filling in missing values

```

missing_tidal_ranges <- tide_hour_date %>%
  mutate(date2 = as.character(date)) %>%
  mutate(time_hms = hms::as_hms(ifelse(nchar(time) <= 5,
                                       paste0(time, ":00"), time))) %>%
  filter(time_hms >= hms::hms(hours = 5) &
         time_hms <= hms::hms(hours = 21)) %>%
  mutate(time_period = case_when(
    time_hms >= hms::hms(hours = 5) &
    time_hms < hms::hms(hours = 8) ~ "Early AM",
    time_hms >= hms::hms(hours = 8) &
    time_hms < hms::hms(hours = 11) ~ "Mid AM",
    time_hms >= hms::hms(hours = 11) &
    time_hms < hms::hms(hours = 13) ~ "Midday",
    time_hms >= hms::hms(hours = 13) &
    time_hms < hms::hms(hours = 15) ~ "Early Afternoon",
    time_hms >= hms::hms(hours = 15) &
    time_hms < hms::hms(hours = 17) ~ "Afternoon",
    time_hms >= hms::hms(hours = 17) &
    time_hms < hms::hms(hours = 19) ~ "Evening",
    TRUE ~ "Night")) %>%
  rename(ControlStn = station) %>%

```

```

inner_join(track_times %>%
  filter(!trip_date %in% tide_hour_filtered$trip_date) %>%
  mutate(date2 = as.character(date),
    start_time = hms::as.hms(
      parse_date_time(if_else(is.na(start_time),
        "08:00:00", start_time), "HMS")),
    end_time = hms::as.hms(
      parse_date_time(if_else(is.na(end_time),
        "18:30:00", end_time),
        "HMS")),
    time_period2 = case_when(start_time >= hms::hms(hours = 5) &
      end_time < hms::hms(hours = 8) ~ "Early AM",
      start_time >= hms::hms(hours = 8) &
      end_time < hms::hms(hours = 11) ~ "Mid AM",
      start_time >= hms::hms(hours = 11) &
      end_time < hms::hms(hours = 13) ~ "Midday",
      start_time >= hms::hms(hours = 13) &
      end_time < hms::hms(hours = 15) ~ "Early Afternoon",
      start_time >= hms::hms(hours = 15) &
      end_time < hms::hms(hours = 17) ~ "Afternoon",
      start_time >= hms::hms(hours = 17) &
      end_time < hms::hms(hours = 19) ~ "Evening",
      TRUE ~ "Night")) %>%
    distinct(date2, time_period2, trip_date, ControlStn, RR),
    by = c("date2", "ControlStn")) %>%
  filter(time_period == time_period2) %>%
  group_by(trip_date, date2, time_period) %>%
  summarise(mean_level = mean(water_level * RR, na.rm=T), .groups="drop") %>%
  ungroup() %>%
  select(-c(time_period)) %>%
  rename(date = date2)

length(unique(missing_tidal_ranges$trip_date))

```

```
## [1] 3960
```

```

missing_tidal_ranges_wider <- tide_hour_date %>%
  mutate(date2 = as.character(date)) %>%
  mutate(time_hms = hms::as_hms(ifelse(nchar(time) <= 5,
    paste0(time, ":00"), time))) %>%
  filter(time_hms >= hms::hms(hours = 5) &
    time_hms <= hms::hms(hours = 21)) %>%
  mutate(time_period = case_when(
    time_hms >= hms::hms(hours = 5) &
    time_hms < hms::hms(hours = 8) ~ "Early AM",
    time_hms >= hms::hms(hours = 8) &
    time_hms < hms::hms(hours = 11) ~ "Mid AM",
    time_hms >= hms::hms(hours = 11) &
    time_hms < hms::hms(hours = 13) ~ "Midday",
    time_hms >= hms::hms(hours = 13) &
    time_hms < hms::hms(hours = 15) ~ "Early Afternoon",
    time_hms >= hms::hms(hours = 15) &
    time_hms < hms::hms(hours = 17) ~ "Afternoon",

```

```

time_hms >= hms::hms(hours = 17) &
time_hms < hms::hms(hours = 19) ~ "Evening",
TRUE ~ "Night")) %>%
rename(ControlStn = station) %>%
inner_join(track_times %>%
  filter(!trip_date %in% tide_hour_filtered$trip_date) %>%
  filter(!trip_date %in% missing_tidal_ranges$trip_date) %>%
  mutate(date2 = as.character(date),
    start_time = hms::as.hms(
      parse_date_time(if_else(is.na(start_time),
        "08:00:00", start_time), "HMS")),
    end_time = hms::as.hms(
      parse_date_time(if_else(is.na(end_time),
        "18:30:00", end_time),
        "HMS")),
    time_period2 = case_when(start_time >= hms::hms(hours = 5) &
      end_time < hms::hms(hours = 8) ~ "Early AM",
      start_time >= hms::hms(hours = 8) &
      end_time < hms::hms(hours = 11) ~ "Mid AM",
      start_time >= hms::hms(hours = 11) &
      end_time < hms::hms(hours = 13) ~ "Midday",
      start_time >= hms::hms(hours = 13) &
      end_time < hms::hms(hours = 15) ~ "Early Afternoon",
      start_time >= hms::hms(hours = 15) &
      end_time < hms::hms(hours = 17) ~ "Afternoon",
      start_time >= hms::hms(hours = 17) &
      end_time < hms::hms(hours = 19) ~ "Evening",
      TRUE ~ "Night")) %>%
  distinct(date2, time_period2, trip_date, ControlStn, RR),
  by = c("date2", "ControlStn")) %>%
filter(time_period == "Afternoon") %>%
group_by(trip_date, date2, time_period) %>%
summarise(mean.level = mean(water.level*RR, na.rm=T), .groups="drop") %>%
ungroup() %>%
select(-c(time_period)) %>%
rename(date = date2) %>%
rbind(missing_tidal_ranges)

updated_survey_levels <- tide_hour_filtered %>%
  full_join(track_times,
    by = c("trip_date", "date")) %>%
  left_join(missing_tidal_ranges_wider %>%
    distinct() %>%
    mutate(date = as.Date(date)), by = c("trip_date", "date")) %>%

  mutate(survey_tide = ifelse((is.na(survey_tide) | is.nan(survey_tide)) &
    !is.na(mean.level),
    mean.level, survey_tide)) %>%
  mutate(survey_tide = ifelse(is.nan(survey_tide), NA, survey_tide)) %>%
  select(-mean.level) %>%
  distinct()

```

```

missing_tide <- updated_survey_levels %>%
  filter(is.na(survey_tide)) %>%
  mutate(date2 = as.Date(date)) %>%
  mutate(month_year = format(date2, "%Y-%m")) %>%
  group_by(month_year)

tide_tracks <- track_transformed %>%
  full_join(updated_survey_levels, by = c("trip_date", "date")) %>%
  st_drop_geometry() %>%
  distinct(trip_date, date, survey_tide) %>%
  filter(!is.na(survey_tide))

all_tracks <- track_transformed %>%
  filter(!trip_date %in% tide_tracks$trip) %>%
  select(trip_date, date) %>%
  mutate(survey_tide = NA) %>%
  distinct() %>%
  st_drop_geometry() %>%
  rbind(tide_tracks)

```

Save

```

write_csv(tidal_obs_level, "GIS_COVS/tidal_obs_level.csv")

saveRDS(all_tracks, "GIS_COVS/tidal_obs_level.rds")

```